17 · Route Guards and Lazy Loading

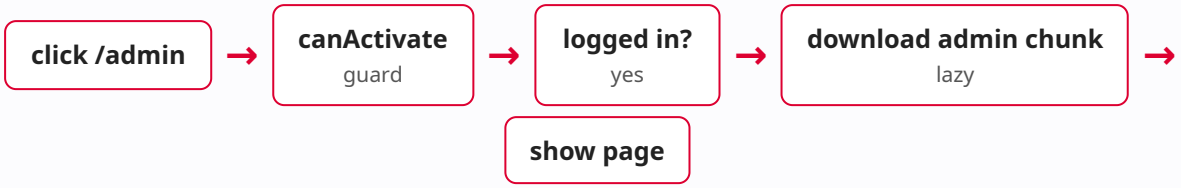
Intermediate Angular learning notes

a. Definition

A **route guard** is a function that decides whether a navigation is allowed. `canActivate` runs before a route is entered; it returns `true` to allow, `false` to block, or a `UrlTree` to **redirect** (for example to a login page). Guards are plain functions that can use `inject()`.

Lazy loading means the code of a page is downloaded only when the user first visits it, using `loadComponent` (or `loadChildren`). This makes the first load smaller and faster.

b. Diagram



Navigation passes through the guard; the lazy chunk downloads only after approval

c. Syntax

```
export const authGuard: CanActivateFn = () => {
  const ok = inject(AuthService).loggedIn();
  return ok ? true : inject(Router).createUrlTree(['/login']);
};

{ path: 'admin',
  canActivate: [authGuard],
  loadComponent: () => import('./admin.component').then(m => m.AdminComponent) }
```

d. Example Program

`Programs/17_Guards_and_Lazy_Loading/guarded-app.component.ts`

```
import { ApplicationConfig, Component, inject, Injectable, signal } from '@angular/core';
import { CanActivateFn, provideRouter, Router, RouterLink, RouterOutlet, Routes,
withHashLocation } from '@angular/router';

@Injectable({ providedIn: 'root' })
export class AuthService {
  loggedIn = signal(false);
}

// Functional guard: true = allow, UrlTree = redirect
export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);
  return auth.loggedIn() ? true : router.createUrlTree(['/login']);
};

@Component({ selector: 'app-home', template: `<h2>Home</h2><p>Public page.</p>` })
```

```
export class HomeComponent {}

@Component({
  selector: 'app-login',
  template: `
    <h2>Login required</h2>
    <button id="login" (click)="login()">Log in</button>
  `,
})
export class LoginComponent {
  private auth = inject(AuthService);
  private router = inject(Router);

  login() {
    this.auth.loggedIn.set(true);
    this.router.navigate(['/admin']);
  }
}

const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'login', component: LoginComponent },
  {
    path: 'admin',
    canActivate: [authGuard],
    // Lazy loaded: admin.component.ts becomes a separate JavaScript chunk
    loadComponent: () => import('./admin.component').then((m) => m.AdminComponent),
  },
];

export const appConfig: ApplicationConfig = {
  providers: [provideRouter(routes, withHashLocation())],
};

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, RouterLink],
  template: `
    <nav>
      <a routerLink="/">Home</a> |
      <a routerLink="/admin">Admin</a>
    </nav>
    <router-outlet />
  `,
})
export class AppComponent {}
```

More programs in the same folder: `admin.component.ts`

e. Output

Rendered in the browser (text on screen, after the UI steps below):

1. Click **Admin** while logged out - the guard redirects to Login.
2. Click **Log in** - you are taken to the lazy loaded Admin page.

```
--- Admin link while logged out (redirected) ---
Home | Admin
Login required
Log in

--- After logging in ---
Home | Admin
Admin Panel
This page was lazy loaded after the guard allowed you in.
```

f. Real-World Example (English + தமிழ்)

English

At a temple special-darshan entrance the staff checks your ticket (guard). Prasadam for each shrine is cooked only when devotees arrive there (lazy loading).

தமிழ்

கோயிலில் சிறப்பு தரிசனத்துக்குச் செல்லும்போது வாசலில் உள்ள ஊழியர் உங்கள் டிக்கெட்டைப் பார்ப்பார். டிக்கெட் இருந்தால் உள்ளே அனுமதி; இல்லையென்றால் 'டிக்கெட் கவுண்டருக்குப் போங்கள்' என்று திருப்பி அனுப்புவார். அவர்தான் **route guard**; கவுண்டருக்கு அனுப்புவது = `UrlTree` redirect.

ஒவ்வொரு சன்னதிக்கும் உரிய பிரசாதம் அந்தச் சன்னதிக்கு பக்தர் வந்த பிறகே தயாரிக்கப்பட்டால், சமையல்கூடம் வீணாகாது. அதுபோல **lazy loading** - பயனர் அந்தப் பக்கத்துக்குப் போனால் மட்டுமே அதன் code பதிவிறக்கம் ஆகும்.

g. Pros

- Smaller first load
- Central access control
- Redirects with UrlTree

h. Cons

- Guards are not real security
- Lazy chunks add a small delay on first visit

i. Advanced / New Features

- Functional guards (`CanActivateFn`)
- `canMatch`
- `@defer` for lazy template parts
- Prefetching with `withPreloading`

j. Common Mistakes

- Returning `false` from a guard without redirecting, leaving the user on a blank screen.
- Relying on guards for security. They only hide UI; the server must still check permissions.
- Using eager `component:` for big rarely-used pages instead of `loadComponent` .
- Calling `inject()` outside the guard's synchronous body (for example after `await`).

k. Interview Questions

Q1. What is a route guard?
A function that controls navigation: `canActivate`, `canDeactivate`, `canMatch`, `resolve` etc.

Q2. How do you redirect from a guard?
Return a `UrlTree` created with `router.createUrlTree([...])`.

Q3. What is lazy loading?
Loading route code on demand through dynamic `import()`, reducing the initial bundle size.

Q4. `canActivate` vs `canMatch`?
`canActivate` runs after the route matches; `canMatch` decides whether the route can match at all (and so whether lazy code is loaded).

l. Practice Exercises

1. Add a `canDeactivate` guard that asks for confirmation when leaving a form with unsaved changes.
2. Make a `roleGuard` that allows only users whose role is 'admin'.
3. Lazy load a `Reports` page and open the browser Network tab to see the extra chunk download.